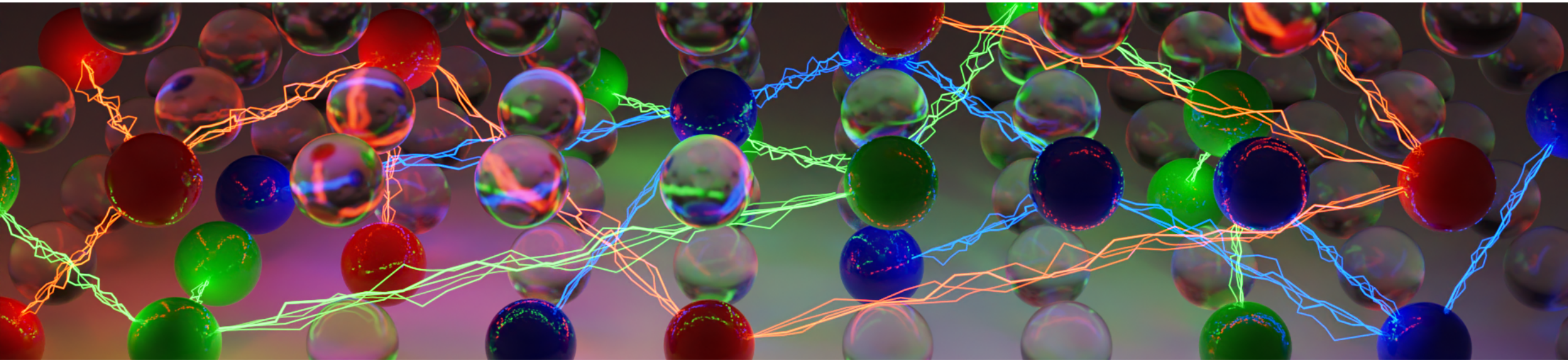
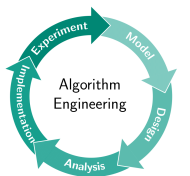




Practical SAT Solving

Lecture 3 – Elementary SAT Solving Algorithms

Ashlin Iser, Dominik Schreiber | May 5, 2025



SAtRes
Scalable Automated Reasoning

Overview

Recap. Lecture 2

- Tractable Subclasses
- Constraint Encodings
- Encoding Techniques

Today's Topics: Elementary SAT Algorithms

- Local Search
- Resolution
- DP Algorithm
- DPLL Algorithm

Stochastic Local Search (SLS)

Minimize the Number of Unsatisfied Clauses

Start with a **random** complete variable assignment α :

0	0	1	0	1	1	0	0	1	1	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Repeatedly **flip** variables in α to decrease the number of unsatisfied clauses:

0	0	1	0	1	0	0	0	1	1	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stochastic Local Search (SLS)

Properties of SLS Algorithms

Local search algorithms are **incomplete**: They **cannot show unsatisfiability**!

Challenges:

- Which variable should be flipped next?

Stochastic Local Search (SLS)

Properties of SLS Algorithms

Local search algorithms are **incomplete**: They **cannot show unsatisfiability**!

Challenges:

- Which variable should be flipped next?
 - select variable from an **unsatisfied clause**
 - select variable that **maximizes** the number of satisfied clauses
- How to avoid getting stuck in **local minima**?

Stochastic Local Search (SLS)

Properties of SLS Algorithms

Local search algorithms are **incomplete**: They **cannot show unsatisfiability**!

Challenges:

- Which variable should be flipped next?
 - select variable from an **unsatisfied clause**
 - select variable that **maximizes** the number of satisfied clauses
- How to avoid getting stuck in **local minima**?
 - **Randomization**!

Classic SLS Algorithms

GSAT

Greedy local search algorithm

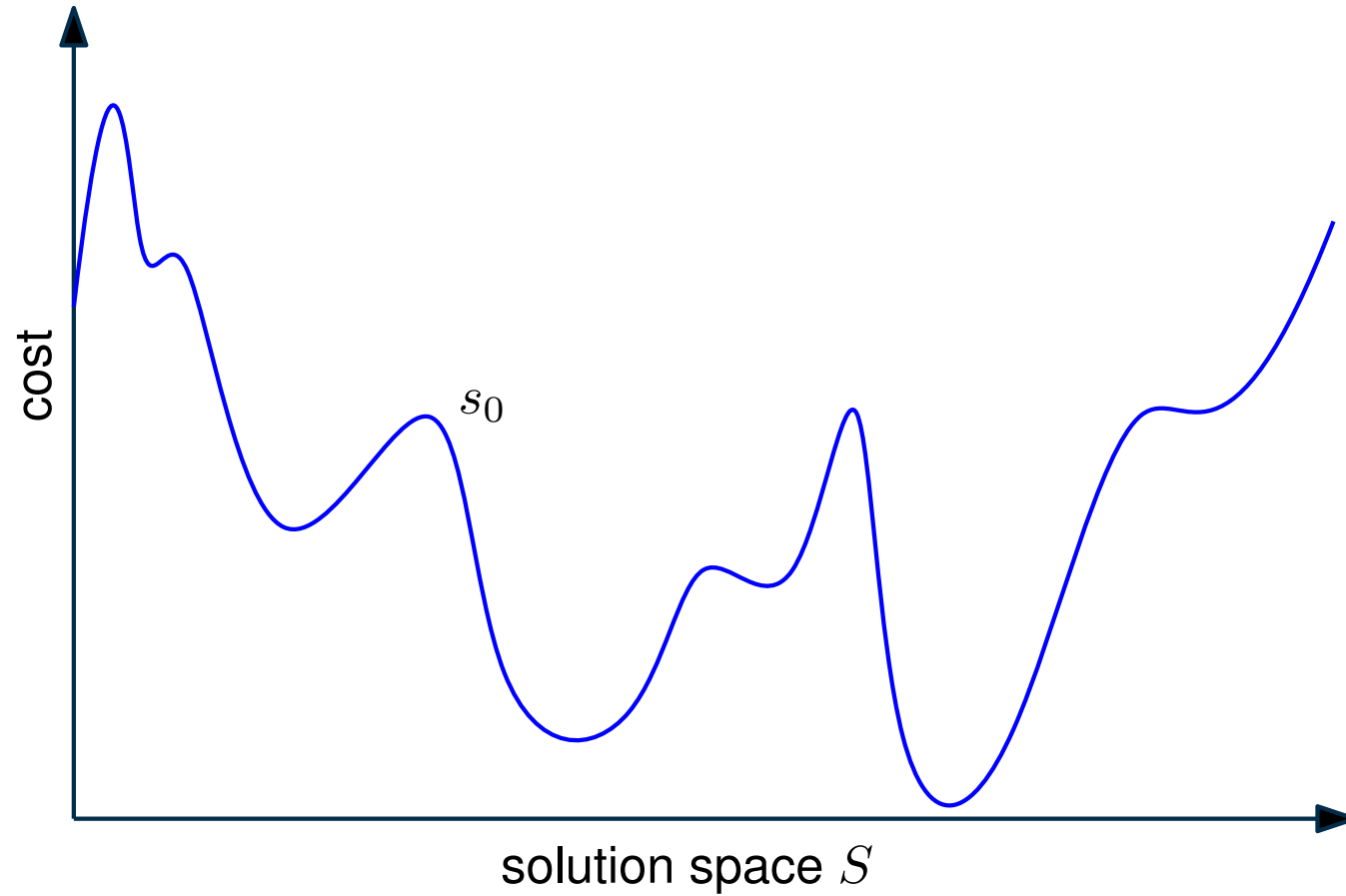
Algorithm: GSAT

Input: ClauseSet S

Output: Assignment α , or Nothing

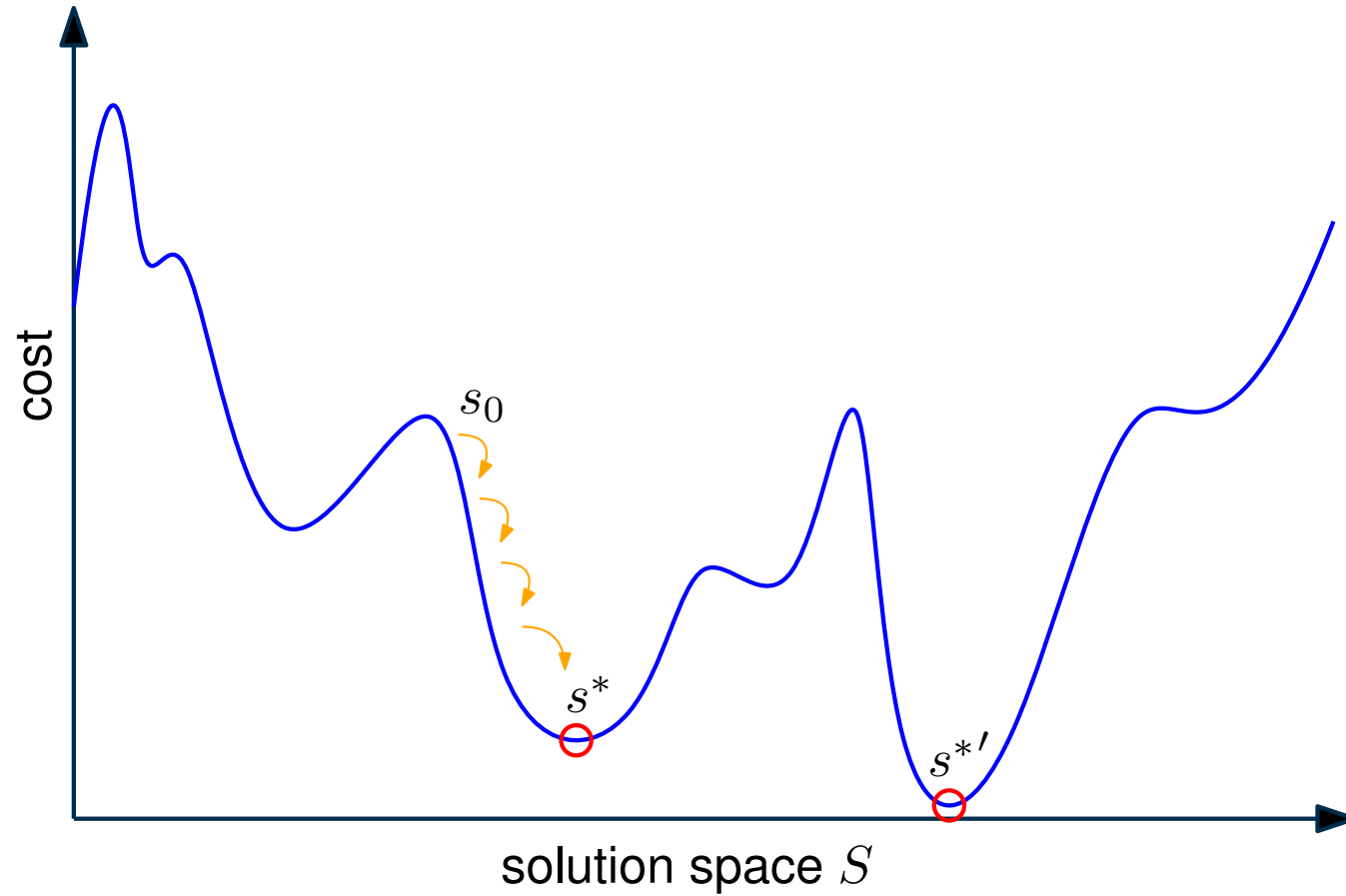
```
1 for  $i = 1$  to  $MAX\_TRIES$  do
2    $\alpha$  = random-assignment to variables in  $S$ 
3   for  $j = 1$  to  $MAX\_FLIPS$  do
4     if  $\alpha$  satisfies all clauses in  $S$  then return  $\alpha$ 
5      $x$  = variable that produces least number of unsatisfied clauses
        when flipped
6     flip  $x$ 
7 return Nothing                                // no solution found
```

SLS: Local Minima



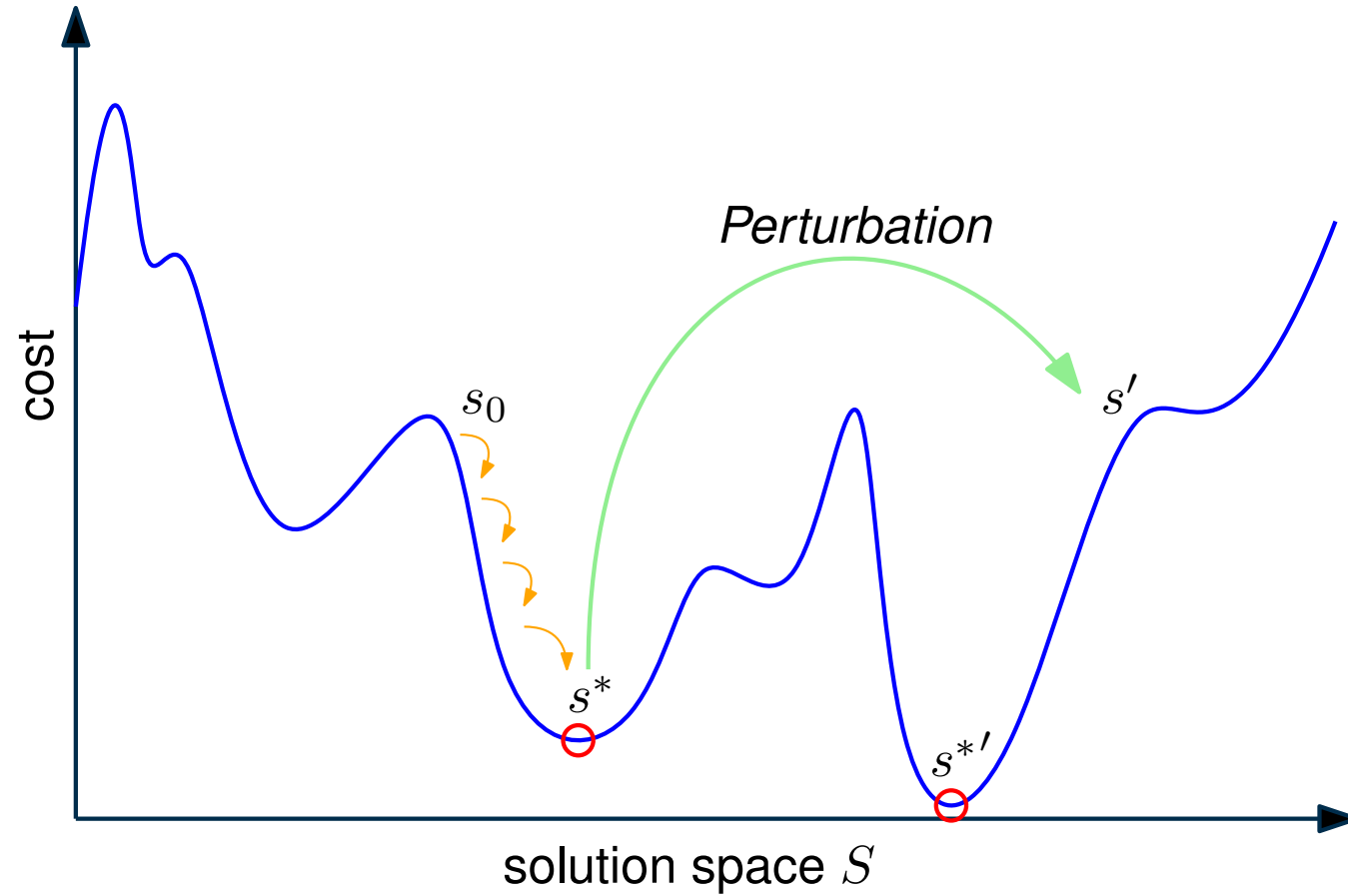
[Illustration Adapted from: Alan Mackworth, UBC, Canada]

SLS: Local Minima



[Illustration Adapted from: Alan Mackworth, UBC, Canada]

SLS: Local Minima



[Illustration Adapted from: Alan Mackworth, UBC, Canada]

Classic SLS Algorithms

WalkSAT

Variant of GSAT

Try to avoid local minima by introducing *random noise*.

Algorithm: WalkSAT(S)

```
1 for  $i = 1$  to  $MAX\_TRIES$  do
2    $\alpha$  = random-assignment to variables in  $S$ 
3   for  $j = 1$  to  $MAX\_FLIPS$  do
4     if  $\alpha$  satisfies all clauses in  $S$  then return  $\alpha$ 
5      $C$  = random unsatisfied clause in  $S$ 
6     if by flipping an  $x \in C$  no new unsatisfied clauses emerges
7       then flip  $x$ 
8     else with probability  $p$  flip an  $x \in C$  at random
9       otherwise, flip a variable that changes the least number of
10        clauses from satisfied to unsatisfied
9 return Nothing // no solution found
```

SLS: Important Notions

Consider a flip taking α to α'

breakcount number of clauses satisfied in α , but not satisfied in α'

makecount number of clauses not satisfied in α , but satisfied in α'

diffscore # unsatisfied clauses in α – # unsatisfied clauses in α'

Typically, breakcount, makecount, and/or diffscore are used to select the variable to flip.

SLS: Important Notions

Consider a flip taking α to α'

breakcount number of clauses satisfied in α , but not satisfied in α'

makecount number of clauses not satisfied in α , but satisfied in α'

diffscore # unsatisfied clauses in α – # unsatisfied clauses in α'

Typically, breakcount, makecount, and/or diffscore are used to select the variable to flip.

Recap using new nomenclature

GSAT select variable with highest diffscore

WalkSAT select variable with minimal breakcount

Stochastic Local Search (SLS)

Legacy of SLS

- Extremely **successful and popular** in early days of SAT
 - SLS outperformed early resolution-based solvers, e.g., based on DP or DPLL
 - for example, state of the art engine for **automated planning** in the 90s

Stochastic Local Search (SLS)

Legacy of SLS

- Extremely **successful and popular** in early days of SAT
 - SLS outperformed early resolution-based solvers, e.g., based on DP or DPLL
 - for example, state of the art engine for **automated planning** in the 90s
- Today, sophisticated resolution-based systematic search solvers dominate in most practical applications
 - Faster, more reliable, and **complete!**

Stochastic Local Search (SLS)

Legacy of SLS

- Extremely **successful and popular** in early days of SAT
 - SLS outperformed early resolution-based solvers, e.g., based on DP or DPLL
 - for example, state of the art engine for **automated planning** in the 90s
- Today, sophisticated resolution-based systematic search solvers dominate in most practical applications
 - Faster, more reliable, and **complete!**
- Still useful as a component in more complex solvers
 - Part of (parallel) **algorithm portfolios**
 - Control **branching heuristics** in complete search algorithms
 - Detection of autarkies in **formula simplification** algorithms
 - In combination with complete solvers for optimization problems (e.g., **MaxSAT**)

Recap

Elementary Algorithms

- Local Search
 - Examples: GSAT, WalkSAT
 - Terminology: breakcount, makecount, diffscore

Recap

Elementary Algorithms

- Local Search
 - Examples: GSAT, WalkSAT
 - Terminology: breakcount, makecount, diffscore

Next Up

- Resolution

Resolution

The Resolution Rule

$$\frac{P_1 \cup \{x\} \quad P_2 \cup \{\neg x\}}{P_1 \cup P_2}$$

Resolution is a logical **inference rule**, which infers a **conclusion** (resolvent) from given **premises** (input clauses).

Resolution

The Resolution Rule

$$\frac{P_1 \cup \{x\} \quad P_2 \cup \{\neg x\}}{P_1 \cup P_2}$$

Resolution is a logical **inference rule**, which infers a **conclusion** (resolvent) from given **premises** (input clauses).

Example (Resolution)

$$\{x_1, x_3, \neg x_7\}, \{\neg x_1, x_2\} \vdash \{x_3, \neg x_7, x_2\}$$

$$\{x_4, x_5\}, \{\neg x_5\} \vdash \{x_4\}$$

(Fact)

$$\{x_1, x_2\}, \{\neg x_1, \neg x_2\} \vdash \{x_1, \neg x_1\}$$

(Tautological Resolvent)

$$\{x_1\}, \{\neg x_1\} \vdash \{\}$$

(Empty Clause)

Resolution

Theorem: Resolution is sound

Given a CNF formula F with two resolvable clauses $C_1, C_2 \subseteq F$ with resolvent $R(C_1, C_2)$, the following holds:

$$F \equiv F \wedge R(C_1, C_2)$$

Resolution

Theorem: Resolution is sound

Given a CNF formula F with two resolvable clauses $C_1, C_2 \subseteq F$ with resolvent $R(C_1, C_2)$, the following holds:

$$F \equiv F \wedge R(C_1, C_2)$$

Proof

Let $C_1 := \{x\} \cup P_1$ and $C_2 := \{\neg x\} \cup P_2$ such that $R(C_1, C_2) = P_1 \cup P_2 =: D$.

Soundness: $F \vdash F \wedge D \implies F \models F \wedge D$

Resolution

Theorem: Resolution is sound

Given a CNF formula F with two resolvable clauses $C_1, C_2 \subseteq F$ with resolvent $R(C_1, C_2)$, the following holds:

$$F \equiv F \wedge R(C_1, C_2)$$

Proof

Let $C_1 := \{x\} \cup P_1$ and $C_2 := \{\neg x\} \cup P_2$ such that $R(C_1, C_2) = P_1 \cup P_2 =: D$.

Soundness: $F \vdash F \wedge D \implies F \models F \wedge D$

Any satisfying assignment ϕ of F satisfies both C_1 and C_2 . If ϕ satisfies x , then it satisfies some literal in P_2 . Otherwise, ϕ satisfies $\neg x$ and thus satisfies some literal in P_1 . As such, ϕ also satisfies D .

Resolution

Theorem: Resolution is sound

Given a CNF formula F with two resolvable clauses $C_1, C_2 \subseteq F$ with resolvent $R(C_1, C_2)$, the following holds:

$$F \equiv F \wedge R(C_1, C_2)$$

Proof

Let $C_1 := \{x\} \cup P_1$ and $C_2 := \{\neg x\} \cup P_2$ such that $R(C_1, C_2) = P_1 \cup P_2 =: D$.

Soundness: $F \vdash F \wedge D \implies F \models F \wedge D$

Any satisfying assignment ϕ of F satisfies both C_1 and C_2 . If ϕ satisfies x , then it satisfies some literal in P_2 . Otherwise, ϕ satisfies $\neg x$ and thus satisfies some literal in P_1 . As such, ϕ also satisfies D .

Equivalence: $F \vdash F \wedge D \implies F \wedge D \models F$

Resolution

Theorem: Resolution is sound

Given a CNF formula F with two resolvable clauses $C_1, C_2 \subseteq F$ with resolvent $R(C_1, C_2)$, the following holds:

$$F \equiv F \wedge R(C_1, C_2)$$

Proof

Let $C_1 := \{x\} \cup P_1$ and $C_2 := \{\neg x\} \cup P_2$ such that $R(C_1, C_2) = P_1 \cup P_2 =: D$.

Soundness: $F \vdash F \wedge D \implies F \models F \wedge D$

Any satisfying assignment ϕ of F satisfies both C_1 and C_2 . If ϕ satisfies x , then it satisfies some literal in P_2 . Otherwise, ϕ satisfies $\neg x$ and thus satisfies some literal in P_1 . As such, ϕ also satisfies D .

Equivalence: $F \vdash F \wedge D \implies F \wedge D \models F$

Since D does not introduce new variables, $F \wedge D$ can not have more satisfying assignments than F .

Resolution

Resolution is sound and Refutation-complete

- If we manage to infer the **empty clause** from a CNF formula F , then F is unsatisfiable. (sound)
- If F is unsatisfiable, then there **exists a refutation by resolution**. (refutation-complete)
- **Not all possible** consequences of F can be derived by resolution. (“only” refutation-complete)

Resolution Proof

A **resolution proof** for F is a sequence of clauses $\langle C_1, C_2, \dots, C_{k-1}, C_k = \emptyset \rangle$ where each C_i is either an original clause of F or a resolvent of two earlier clauses.

Example (Resolution Proof)

$$F = \{x_1, x_2\}, \{\neg x_1, x_2\}, \{x_1, \neg x_2\}, \{\neg x_1, \neg x_2\}$$

(Formula)
(Refutation)

Resolution

Resolution is sound and Refutation-complete

- If we manage to infer the **empty clause** from a CNF formula F , then F is unsatisfiable. (sound)
- If F is unsatisfiable, then there **exists a refutation by resolution**. (refutation-complete)
- **Not all possible** consequences of F can be derived by resolution. (“only” refutation-complete)

Resolution Proof

A **resolution proof** for F is a sequence of clauses $\langle C_1, C_2, \dots, C_{k-1}, C_k = \emptyset \rangle$ where each C_i is either an original clause of F or a resolvent of two earlier clauses.

Example (Resolution Proof)

$$\begin{aligned} F &= \{x_1, x_2\}, \{\neg x_1, x_2\}, \{x_1, \neg x_2\}, \{\neg x_1, \neg x_2\} && \text{(Formula)} \\ &\equiv \{x_1, x_2\}, \{\neg x_1, x_2\}, \{x_1, \neg x_2\}, \{\neg x_1, \neg x_2\}, \{x_2\}, \{\neg x_2\}, \{\} && \text{(Refutation)} \end{aligned}$$

Saturation Algorithm

Algorithm: Saturation Algorithm

Input: CNF formula F

Output: {SAT, UNSAT}

```
1 while true do
2    $R := \text{resolveAll}(F)$ 
3   if  $R \cap F \neq R$  then  $F := F \cup R$ 
4   else break
5 if  $\perp \in F$  then return UNSAT
6 else return SAT
```

Saturation Algorithm

Algorithm: Saturation Algorithm

Input: CNF formula F

Output: {SAT, UNSAT}

```
1 while true do
2    $R := \text{resolveAll}(F)$ 
3   if  $R \cap F \neq R$  then  $F := F \cup R$ 
4   else break
5 if  $\perp \in F$  then return UNSAT
6 else return SAT
```

Properties

- sound and complete – always terminates and answers correctly
- exponential time and space complexity

Unit Propagation

Unit Resolution

Resolution where at least one of the resolved clauses is a **unit clause**, i.e., has size one.

Example (Unit Resolution)

$$R((x_1 \vee x_7 \vee \neg x_2 \vee x_4), (x_2)) = (x_1 \vee x_7 \vee x_4)$$

Unit Propagation

Unit Resolution

Resolution where at least one of the resolved clauses is a **unit clause**, i.e., has size one.

Example (Unit Resolution)

$$R((x_1 \vee x_7 \vee \neg x_2 \vee x_4), (x_2)) = (x_1 \vee x_7 \vee x_4)$$

Unit Propagation

Apply unit resolution until fixpoint is reached.

Example (Unit Propagation)

Usually, we are only interested in the inferred facts (unit clauses) and conflicts (empty clauses).

$$\{x_1, x_2, x_3\}, \{x_1, \neg x_2\}, \{\neg x_1\} \vdash_1 \{\neg x_2\}, \{x_3\}$$

Recap

Elementary Algorithms

- Local Search
 - Examples: GSAT, WalkSAT
 - Terminology: breakcount, makecount, diffscore
- Resolution
 - Soundness and Completeness
 - Saturation Algorithm (Exponential Complexity)
 - Unit Propagation

Recap

Elementary Algorithms

- Local Search
 - Examples: GSAT, WalkSAT
 - Terminology: breakcount, makecount, diffscore
- Resolution
 - Soundness and Completeness
 - Saturation Algorithm (Exponential Complexity)
 - Unit Propagation

Next Up

Improving upon saturation-based resolution: Davis Putnam (DP) Algorithm

Davis-Putnam Algorithm (Davis & Putnam, 1960)

Presented in 1960 as a SAT procedure for first-order logic.

Deduction Rules of DP Algorithm

- **Unit Resolution:** If there is a unit clause $C = \{\bar{x}\} \in F$, simplify all other clauses containing x
- **Pure Literal Elimination:** If a literal x never occurs negated in F , add clause $\{x\}$ to F
- **Case Splitting:** Put F in the form $(A \vee x) \wedge (B \vee \neg x) \wedge R$, where A , B , and R are clause sets free of x . Replace F by the clausification of $(A \vee B) \wedge R$

Apply above deduction rules (prioritizing rules 1 and 2) until one of the following situations occurs:

- $F = \emptyset \rightarrow \text{SAT}$
- $\emptyset \in F \rightarrow \text{UNSAT}$

Davis-Putnam Algorithm

Example (DP Algorithm)

$$F = \{\{x, y, \neg z, u\}, \{\neg x, y, u\}, \{x, \neg y, \neg z\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}, \{\neg x, \neg y, u\}\} \quad (\text{Split by } x)$$

Davis-Putnam Algorithm

Example (DP Algorithm)

$$\begin{aligned} F &= \{\{x, y, \neg z, u\}, \{\neg x, y, u\}, \{x, \neg y, \neg z\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}, \{\neg x, \neg y, u\}\} && \text{(Split by } x) \\ A &= \{\{y, \neg z, u\}, \{\neg y, \neg z\}\} \quad B = \{\{y, u\}, \{\neg y, u\}\} \quad R = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} && ((A \vee B) \wedge R) \end{aligned}$$

Davis-Putnam Algorithm

Example (DP Algorithm)

$$\begin{aligned} F &= \{\{x, y, \neg z, u\}, \{\neg x, y, u\}, \{x, \neg y, \neg z\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}, \{\neg x, \neg y, u\}\} && \text{(Split by } x) \\ A &= \{\{y, \neg z, u\}, \{\neg y, \neg z\}\} \quad B = \{\{y, u\}, \{\neg y, u\}\} \quad R = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} && ((A \vee B) \wedge R) \\ F_1 &= \{\{y, \neg z, u\}, \{\neg y, \neg z, u\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} && \text{(Split by } y) \end{aligned}$$

Davis-Putnam Algorithm

Example (DP Algorithm)

$$F = \{\{x, y, \neg z, u\}, \{\neg x, y, u\}, \{x, \neg y, \neg z\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}, \{\neg x, \neg y, u\}\} \quad (\text{Split by } x)$$

$$A = \{\{y, \neg z, u\}, \{\neg y, \neg z\}\} \quad B = \{\{y, u\}, \{\neg y, u\}\} \quad R = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad ((A \vee B) \wedge R)$$

$$F_1 = \{\{y, \neg z, u\}, \{\neg y, \neg z, u\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad (\text{Split by } y)$$

$$A_1 = \{\{\neg z, u\}\} \quad B_1 = \{\{\neg z, u\}\} \quad R_1 = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad ((A_1 \vee B_1) \wedge R_1)$$

Davis-Putnam Algorithm

Example (DP Algorithm)

$$F = \{\{x, y, \neg z, u\}, \{\neg x, y, u\}, \{x, \neg y, \neg z\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}, \{\neg x, \neg y, u\}\} \quad (\text{Split by } x)$$

$$A = \{\{y, \neg z, u\}, \{\neg y, \neg z\}\} \quad B = \{\{y, u\}, \{\neg y, u\}\} \quad R = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad ((A \vee B) \wedge R)$$

$$F_1 = \{\{y, \neg z, u\}, \{\neg y, \neg z, u\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad (\text{Split by } y)$$

$$A_1 = \{\{\neg z, u\}\} \quad B_1 = \{\{\neg z, u\}\} \quad R_1 = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad ((A_1 \vee B_1) \wedge R_1)$$

$$F_2 = \{\{\neg z, u\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad (\text{Split by } z)$$

Davis-Putnam Algorithm

Example (DP Algorithm)

$$F = \{\{x, y, \neg z, u\}, \{\neg x, y, u\}, \{x, \neg y, \neg z\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}, \{\neg x, \neg y, u\}\} \quad (\text{Split by } x)$$

$$A = \{\{y, \neg z, u\}, \{\neg y, \neg z\}\} \quad B = \{\{y, u\}, \{\neg y, u\}\} \quad R = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad ((A \vee B) \wedge R)$$

$$F_1 = \{\{y, \neg z, u\}, \{\neg y, \neg z, u\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad (\text{Split by } y)$$

$$A_1 = \{\{\neg z, u\}\} \quad B_1 = \{\{\neg z, u\}\} \quad R_1 = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad ((A_1 \vee B_1) \wedge R_1)$$

$$F_2 = \{\{\neg z, u\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad (\text{Split by } z)$$

$$A_2 = \{\{v\}, \{\neg v\}\} \quad B_2 = \{\{u\}, \{\neg u\}\} \quad R_2 = \{\} \quad ((A_2 \vee B_2) \wedge R_2)$$

Davis-Putnam Algorithm

Example (DP Algorithm)

$$F = \{\{x, y, \neg z, u\}, \{\neg x, y, u\}, \{x, \neg y, \neg z\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}, \{\neg x, \neg y, u\}\} \quad (\text{Split by } x)$$
$$A = \{\{y, \neg z, u\}, \{\neg y, \neg z\}\} \quad B = \{\{y, u\}, \{\neg y, u\}\} \quad R = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad ((A \vee B) \wedge R)$$

$$F_1 = \{\{y, \neg z, u\}, \{\neg y, \neg z, u\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad (\text{Split by } y)$$
$$A_1 = \{\{\neg z, u\}\} \quad B_1 = \{\{\neg z, u\}\} \quad R_1 = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad ((A_1 \vee B_1) \wedge R_1)$$

$$F_2 = \{\{\neg z, u\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad (\text{Split by } z)$$
$$A_2 = \{\{v\}, \{\neg v\}\} \quad B_2 = \{\{u\}, \{\neg u\}\} \quad R_2 = \{\} \quad ((A_2 \vee B_2) \wedge R_2)$$

$$F_3 = \{\{u, v\}, \{u, \neg v\}, \{\neg u, v\}, \{\neg u, \neg v\}\} \quad (\text{Split by } u)$$

Davis-Putnam Algorithm

Example (DP Algorithm)

$$F = \{\{x, y, \neg z, u\}, \{\neg x, y, u\}, \{x, \neg y, \neg z\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}, \{\neg x, \neg y, u\}\} \quad (\text{Split by } x)$$
$$A = \{\{y, \neg z, u\}, \{\neg y, \neg z\}\} \quad B = \{\{y, u\}, \{\neg y, u\}\} \quad R = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad ((A \vee B) \wedge R)$$

$$F_1 = \{\{y, \neg z, u\}, \{\neg y, \neg z, u\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad (\text{Split by } y)$$
$$A_1 = \{\{\neg z, u\}\} \quad B_1 = \{\{\neg z, u\}\} \quad R_1 = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad ((A_1 \vee B_1) \wedge R_1)$$

$$F_2 = \{\{\neg z, u\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} \quad (\text{Split by } z)$$
$$A_2 = \{\{v\}, \{\neg v\}\} \quad B_2 = \{\{u\}, \{\neg u\}\} \quad R_2 = \{\} \quad ((A_2 \vee B_2) \wedge R_2)$$

$$F_3 = \{\{u, v\}, \{u, \neg v\}, \{\neg u, v\}, \{\neg u, \neg v\}\} \quad (\text{Split by } u)$$
$$A_3 = \{\{v\}, \{\neg v\}\} \quad B_3 = \{\{v\}, \{\neg v\}\} \quad R_3 = \{\} \quad ((A_3 \vee B_3) \wedge R_3)$$

Davis-Putnam Algorithm

Example (DP Algorithm)

$$\begin{aligned} F &= \{\{x, y, \neg z, u\}, \{\neg x, y, u\}, \{x, \neg y, \neg z\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}, \{\neg x, \neg y, u\}\} && \text{(Split by } x) \\ A &= \{\{y, \neg z, u\}, \{\neg y, \neg z\}\} \quad B = \{\{y, u\}, \{\neg y, u\}\} \quad R = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} && ((A \vee B) \wedge R) \\ F_1 &= \{\{y, \neg z, u\}, \{\neg y, \neg z, u\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} && \text{(Split by } y) \\ A_1 &= \{\{\neg z, u\}\} \quad B_1 = \{\{\neg z, u\}\} \quad R_1 = \{\{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} && ((A_1 \vee B_1) \wedge R_1) \\ F_2 &= \{\{\neg z, u\}, \{z, v\}, \{z, \neg v\}, \{\neg z, \neg u\}\} && \text{(Split by } z) \\ A_2 &= \{\{v\}, \{\neg v\}\} \quad B_2 = \{\{u\}, \{\neg u\}\} \quad R_2 = \{\} && ((A_2 \vee B_2) \wedge R_2) \\ F_3 &= \{\{u, v\}, \{u, \neg v\}, \{\neg u, v\}, \{\neg u, \neg v\}\} && \text{(Split by } u) \\ A_3 &= \{\{v\}, \{\neg v\}\} \quad B_3 = \{\{v\}, \{\neg v\}\} \quad R_3 = \{\} && ((A_3 \vee B_3) \wedge R_3) \\ F_4 &= \{\{v\}, \{\neg v\}\} \vdash_1 \{\emptyset\} && \text{(Unit Resolution)} \end{aligned}$$

DP Variant: Bucket Elimination

Bucket Elimination

- Fix order $\prec$ on variables.
- **Bucket:** set of clauses with same $\prec$ -maximal variable
- **Bucket Elimination:** process buckets in decreasing $\prec$ -order
 - resolve all clauses in bucket
 - put resolvents in fitting bucket

Example (Bucket Elimination)

$$F = \{(x, y, \bar{z}, u), (\bar{x}, y, u), (x, \bar{y}, \bar{z}), (z, v), (z, \bar{v}), (\bar{z}, \bar{u}), (\bar{x}, \bar{y}, u)\}$$

$$(x \succ y \succ z \succ u \succ v)$$

Variable	Bucket
x	$(x, y, \bar{z}, u), (\bar{x}, y, u), (x, \bar{y}, \bar{z}), (\bar{x}, \bar{y}, u)$
y	
z	$(z, v), (z, \bar{v}), (\bar{z}, \bar{u})$
u	
v	

DP Variant: Bucket Elimination

Bucket Elimination

- Fix **order** $\prec$ on variables.
- **Bucket**: set of clauses with same $\prec$ -maximal variable
- **Bucket Elimination**: process buckets in decreasing $\prec$ -order
 - resolve all clauses in bucket
 - put resolvents in fitting bucket

Example (Bucket Elimination)

$$F = \{(x, y, \bar{z}, u), (\bar{x}, y, u), (x, \bar{y}, \bar{z}), (z, v), (z, \bar{v}), (\bar{z}, \bar{u}), (\bar{x}, \bar{y}, u)\}$$

$$(x \succ y \succ z \succ u \succ v)$$

Variable	Bucket
x	processed
y	$(y, \bar{z}, u), (\bar{y}, \bar{z}, u)$
z	$(z, v), (z, \bar{v}), (\bar{z}, \bar{u})$
u	
v	

DP Variant: Bucket Elimination

Bucket Elimination

- Fix **order** $\prec$ on variables.
- **Bucket**: set of clauses with same $\prec$ -maximal variable
- **Bucket Elimination**: process buckets in decreasing $\prec$ -order
 - resolve all clauses in bucket
 - put resolvents in fitting bucket

Example (Bucket Elimination)

$$F = \{(x, y, \bar{z}, u), (\bar{x}, y, u), (x, \bar{y}, \bar{z}), (z, v), (z, \bar{v}), (\bar{z}, \bar{u}), (\bar{x}, \bar{y}, u)\}$$

$$(x \succ y \succ z \succ u \succ v)$$

Variable	Bucket
x	processed
y	processed
z	$(z, v), (z, \bar{v}), (\bar{z}, \bar{u}), (\bar{z}, u)$
u	
v	

DP Variant: Bucket Elimination

Bucket Elimination

- Fix order $\prec$ on variables.
- **Bucket:** set of clauses with same $\prec$ -maximal variable
- **Bucket Elimination:** process buckets in decreasing $\prec$ -order
 - resolve all clauses in bucket
 - put resolvents in fitting bucket

Example (Bucket Elimination)

$$F = \{(x, y, \bar{z}, u), (\bar{x}, y, u), (x, \bar{y}, \bar{z}), (z, v), (z, \bar{v}), (\bar{z}, \bar{u}), (\bar{x}, \bar{y}, u)\}$$

$$(x \succ y \succ z \succ u \succ v)$$

Variable	Bucket
x	processed
y	processed
z	processed
u	$(\bar{u}, v), (u, v), (\bar{u}, \bar{v}), (u, \bar{v})$
v	

DP Variant: Bucket Elimination

Bucket Elimination

- Fix **order** $\prec$ on variables.
- **Bucket**: set of clauses with same $\prec$ -maximal variable
- **Bucket Elimination**: process buckets in decreasing $\prec$ -order
 - resolve all clauses in bucket
 - put resolvents in fitting bucket

Example (Bucket Elimination)

$$F = \{(x, y, \bar{z}, u), (\bar{x}, y, u), (x, \bar{y}, \bar{z}), (z, v), (z, \bar{v}), (\bar{z}, \bar{u}), (\bar{x}, \bar{y}, u)\}$$

$$(x \succ y \succ z \succ u \succ v)$$

Variable	Bucket
x	processed
y	processed
z	processed
u	processed
v	$(v), (\bar{v})$

DP: Discussion

Excerpt from Davis' and Putnam's paper

*The superiority of the present procedure over those previously available is indicated in part by the fact that a formula on which **Gilmore's routine for the IBM 704** causes the machine to compute for **21 minutes without obtaining a result** was worked successfully by hand computation using [DP] in 30 minutes.*

- Does DP improve on saturation's average time complexity?

DP: Discussion

Excerpt from Davis' and Putnam's paper

*The superiority of the present procedure over those previously available is indicated in part by the fact that a formula on which **Gilmore's routine for the IBM 704** causes the machine to compute for **21 minutes without obtaining a result** was worked successfully by hand computation using [DP] in 30 minutes.*

- Does DP improve on saturation's average time complexity?
⇒ **yes** — if we split over the right variables

DP: Discussion

Excerpt from Davis' and Putnam's paper

*The superiority of the present procedure over those previously available is indicated in part by the fact that a formula on which **Gilmore's routine for the IBM 704** causes the machine to compute for **21 minutes without obtaining a result** was worked successfully by hand computation using [DP] in 30 minutes.*

- Does DP improve on saturation's average time complexity?
⇒ **yes** — if we split over the right variables
- Does DP avoid saturation's exponential space complexity?

DP: Discussion

Excerpt from Davis' and Putnam's paper

*The superiority of the present procedure over those previously available is indicated in part by the fact that a formula on which **Gilmore's routine for the IBM 704** causes the machine to compute for **21 minutes without obtaining a result** was worked successfully by hand computation using [DP] in 30 minutes.*

- Does DP improve on saturation's average time complexity?
⇒ **yes** — if we split over the right variables
- Does DP avoid saturation's exponential space complexity?
⇒ **no** — **quadratic blowup in size** for eliminating one variable

DPLL Algorithm (Davis et al., 1962)

Davis Putnam Logemann Loveland (DPLL) Algorithm

- DPLL is a backtracking search over partial variable assignments.
- Case splitting over a variable x branches the search over two cases x and $\neg x$: resulting in the simplified formulas $F_{|x=\text{true}}$ and $F_{|x=\text{false}}$
- Simplification rules:
 - **Unit Propagation:** If $\{l\} \in F$, l must be set to true.
 - **Pure Literal Elimination:** If x occurs only positively (or only negatively), it may be fixed to the respective value.

DPLL Algorithm

Algorithm: DPLL(ClauseSet S)

```
1 while  $S$  contains a unit clause  $\{L\}$  do
2   | delete from  $S$  all clauses containing  $L$            // unit-subsumption
3   | delete  $\neg L$  from all clauses in  $S$                // unit-resolution
4 if  $\emptyset \in S$  then return false                   // empty clause
5 while  $S$  contains a pure literal  $L$  do
6   | delete from  $S$  all clauses containing  $L$  // pure literal elimination
7 if  $S = \emptyset$  then return true                     // no clauses
8 choose a literal  $L$  occurring in  $S$                    // case-splitting
9 if  $DPLL(S \cup \{\{L\}\})$  then return true           // first branch
10 else if  $DPLL(S \cup \{\{\neg L\}\})$  then return true // second branch
11 else return false
```

Start with
simplifications;

recurse on
subformulas obtained
by case-splitting;

stop if satisfying
assignment found or
all branches are
unsatisfiable.

DPLL Algorithm with Trail

Algorithm: trailDPLL(ClauseSet S , PartialAssignment α)

```
1 while ( $S, \alpha$ ) contains a unit clause  $\{L\}$  do
2   | add  $\{L = 1\}$  to  $\alpha$                                 // Unit Propagation
3 if a literal is assigned both 0 and 1 in  $\alpha$  then
4   | return false                                          // Conflict
5 if all literals assigned then
6   | return true                                           // Assignment found
7 choose a literal  $L$  not assigned in  $\alpha$  occurring in  $S$  // Case Splitting
8 if trailDPLL( $S, \alpha \cup \{\{L = 1\}\}$ ) then
9   | return true                                           // first branch
10 else if trailDPLL( $S, \alpha \cup \{\{L = 0\}\}$ ) then
11   | return true                                           // second branch
12 else return false
```

(S, α) is the clause set S
as “seen” under partial
assignment α

No explicit pure literal
elimination (too slow for the
benefit it provides)

trailDPLL() leads to efficient
iterative DPLL
implementation

DPLL Algorithm

Properties

- DPLL always terminates

DPLL Algorithm

Properties

- DPLL always terminates
 - Each recursion eliminates one variable
 - Worst case: binary tree search of depth $|V|$
- DPLL is sound and complete
 - If clause set S is SAT,

DPLL Algorithm

Properties

- DPLL always terminates
 - Each recursion eliminates one variable
 - Worst case: binary tree search of depth $|V|$
- DPLL is sound and complete
 - If clause set S is SAT, we eventually find a satisfying α
 - If clause set S is UNSAT,

DPLL Algorithm

Properties

- DPLL always terminates
 - Each recursion eliminates one variable
 - Worst case: binary tree search of depth $|V|$
- DPLL is sound and complete
 - If clause set S is SAT, we eventually find a satisfying α
 - If clause set S is UNSAT, the entire space of (partial) variable assignments is searched (but variable selection still matters!)
- Space complexity:

DPLL Algorithm

Properties

- DPLL **always terminates**
 - Each recursion eliminates one variable
 - Worst case: **binary tree search** of depth $|V|$
- DPLL is **sound and complete**
 - If clause set S is **SAT**, we eventually find a satisfying α
 - If clause set S is **UNSAT**, the entire space of (partial) variable assignments is searched (but **variable selection still matters!**)
- Space complexity: **linear!**
 - Systematic search avoids blowup of “**unfocused**” DP

Recap

Elementary Algorithms

- Local Search
 - Examples: GSAT, WalkSAT
 - Terminology: breakcount, makecount, diffscore
- Resolution
 - Soundness and Completeness
 - Saturation Algorithm (Exponential Complexity)
- DP Algorithm
 - Systematized Resolution
 - Improved Average Time Complexity
- DPLL Algorithm
 - Case Splitting and Unit Propagation
 - Linear Space Complexity